

NIYOJAN

Agentic AI–Powered Strategic Demand Planning Platform

1. Introduction

Demand forecasting is a critical component of modern supply chain and inventory management systems. Accurate predictions of future demand enable organizations to optimize procurement, maintain optimal inventory levels, and reduce operational costs. However, in many real-world systems, forecasting alone is insufficient for effective decision-making.

Traditional machine learning forecasting systems generate numerical predictions but often fail to provide actionable insights. Business planners frequently struggle to interpret model outputs and translate them into operational decisions such as restocking inventory, reducing excess stock, or adjusting procurement strategies.

To address these limitations, this project introduces NIYOJAN, an AI-powered demand planning platform designed to transform predictive analytics into actionable strategic planning.

The system evolves through three progressive intelligence stages:

NIYOJAN V1 — Forecasting System

Focuses on demand prediction using machine learning models.

NIYOJAN V2 — Decision Intelligence System

Integrates forecasting with explainable decision logic and generative AI insights.

NIYOJAN V3 — Agentic AI Strategic Planning Platform

Introduces an Agentic AI Intelligence Hub capable of scenario simulation, contextual reasoning, and strategic procurement planning.

By combining machine learning, rule-based decision logic, generative AI, and agentic reasoning, NIYOJAN provides a comprehensive decision-support platform for inventory planning and supply chain management.

2. Problem Statement

Many existing demand forecasting systems suffer from several key limitations:

1. Forecast-Only Outputs

Traditional systems generate numerical predictions without translating them into operational actions.

2. Lack of Explainability

Business users often struggle to understand why demand forecasts change.

3. Poor Decision Support

Predictions are rarely linked to inventory management decisions.

4. Limited Strategic Planning

Most systems cannot simulate future scenarios or suggest procurement strategies.

5. Lack of Deployable Systems

Many academic solutions remain theoretical and are not implemented as deployable platforms.

These challenges highlight the need for an end-to-end AI-driven demand planning platform capable of transforming predictions into strategic decision intelligence.

3. Objectives

The key objectives of the NIYOJAN platform are:

- Build an end-to-end demand planning system beyond simple forecasting
 - Integrate machine learning forecasting with explainable decision logic
 - Use generative AI to provide human-readable insights
 - Introduce agentic AI for strategic planning and scenario simulation
 - Provide an intuitive dashboard for business users
 - Ensure real-world usability through cloud deployment and containerization
-

4. System Overview

NIYOJAN is a multi-layer AI system that integrates predictive analytics, explainable decision logic, generative AI explanations, and agentic planning.

The platform is designed as a modular architecture consisting of the following layers:

1. User Interaction Layer
2. Application Backend Layer
3. AI Intelligence Layer
4. Agentic AI Planning Layer
5. Data Storage Layer

- 6. Reporting and Output Layer
- 7. Deployment Infrastructure

This layered architecture ensures scalability, modularity, and maintainability.

5. System Architecture

The overall architecture of the NIYOJAN platform follows a structured flow from user interaction to strategic planning output.

High-Level System Flow



This architecture enables the system to process raw data, generate forecasts, classify risks, produce explanations, and create strategic planning outputs.

6. User Interaction Layer

The system is designed for business planners and supply chain managers.

Users interact with the system through a web-based dashboard where they can:

- Upload historical sales data
- View demand forecasts
- Monitor inventory risk alerts
- Request AI-generated insights
- Ask strategic planning questions
- Download executive reports

Example user queries include:

"Which products have the highest inventory risk next month?"

"What happens if demand increases by 15%?"

"What procurement strategy should we follow?"

7. Web Dashboard (Frontend)

The user interface is implemented using modern web technologies.

Technology Stack

- React
- TypeScript
- Vite
- Recharts
- Tailwind CSS

Dashboard Features

- CSV data upload interface
- Demand forecast visualization
- Inventory risk alerts
- KPI dashboards
- AI insight generation panel
- Heatmap visualization
- Report preview and download

The dashboard focuses on decision support rather than raw data visualization.

8. Backend API Layer

The backend of the system is implemented using FastAPI, which acts as the orchestration layer connecting the frontend with AI modules.

Responsibilities

- Authentication and authorization
- Data preprocessing
- Model inference
- Decision logic execution
- Agent orchestration
- Report generation

Example API Endpoints

/auth/login
/forecast
/alerts
/insight
/intelligence
/report
/send-report

Security mechanisms include JWT-based authentication and role-based access control.

9. Forecast Engine (Machine Learning Layer)

The forecasting engine predicts future demand using a deep learning model.

Model Used

Long Short-Term Memory (LSTM)

Why LSTM?

- Captures temporal dependencies in time-series data
- Handles seasonal patterns
- Performs well on sequential forecasting tasks

Input

Historical weekly sales data.

Output

Predicted demand for future time steps.

Forecast Pipeline

Sales Data



Data Preprocessing



Scaling



LSTM Model



Recursive Forecast



The predictions form the foundation for downstream decision analysis.

10. Decision Engine (Explainable AI Layer)

The decision engine converts numerical forecasts into interpretable inventory risk categories.

Risk Classification Rules

Forecast $> 1.2 \times$ current stock $\rightarrow$ High Risk (Restock)

Forecast $< 0.5 \times$ current stock $\rightarrow$ Medium Risk (Overstock)

Else $\rightarrow$ Low Risk (Stable)

Advantages

- Transparent decision logic
- Business-aligned thresholds
- Easy to audit and explain

The output of this layer includes:

- inventory risk classification
 - recommended actions
-

11. Generative AI Insight Engine

Generative AI is used to translate technical outputs into natural language explanations.

Role of GenAI

- Explain demand trends
- Interpret risk alerts
- Suggest business actions
- Improve usability for non-technical users

Key design principle:

GenAI is assistive, not authoritative.

It does not perform calculations or override rule-based decisions.

12. Agentic AI Intelligence Hub (V3)

The most advanced component of NIYOJAN is the Agentic AI Intelligence Hub, which transforms the system into a strategic planning assistant.

This module enables the system to:

- Understand natural language planning questions
- Analyze forecast data
- Simulate demand scenarios
- Recalculate inventory risk
- Generate procurement strategies

The Intelligence Hub is implemented using LangGraph, which allows structured agent workflows.

13. Agent Workflow

The Agentic AI system follows a structured reasoning pipeline.

Execution Flow

User Query



Intent Detection



Routing Decision



Retrieval / Analysis / Simulation / Strategy



Context Aggregation



Strategic LLM Reasoning



Insight Formatting



Executive Planning Output

14. Intent Detection

The system first determines the type of user query.

Possible intents:

- Retrieval
- Analysis
- Simulation
- Strategy

Example:

"What if demand increases by 15%"

→ Simulation Intent

15. Execution Paths

The agent routes the query into one of four execution paths.

Path 1 – Retrieval (RAG)

Used for document-based questions.

Process:

- embed user query
 - similarity search in vector database
 - retrieve relevant text chunks
-

Path 2 – Analysis

Performs descriptive analytics such as:

- volatility calculation
- inventory gap detection
- lead time analysis

These operations are performed using deterministic Python calculations.

Path 3 – Simulation

This path handles scenario analysis.

Example:

"What if demand increases by 15%?"

Calculations performed:

New Demand

= forecast × (1 + percentage)

Safety Stock

$= Z \times \text{std_dev} \times \sqrt{\text{lead_time}}$

Reorder Point

$= \text{demand} \times \text{lead_time} + \text{safety stock}$

Inventory Gap

$= \text{reorder point} - \text{stock}$

Risk levels are recalculated based on these values.

Path 4 – Strategy Preparation

This stage aggregates all relevant signals:

- risk metrics
- simulation results
- volatility data
- retrieved knowledge

These inputs form the strategic context for the reasoning engine.

16. Strategic LLM Reasoning

The LLM synthesizes structured inputs to generate planning recommendations.

Important constraint:

The LLM does not perform calculations.

It only performs reasoning based on structured inputs.

Outputs include:

- procurement strategy
 - SKU prioritization
 - operational planning recommendations
-

17. Insight Formatting

The reasoning output is converted into structured executive insights.

Output format includes:

Executive Summary
High Risk Products
Key Risk Drivers
Recommended Actions
4-Week Procurement Plan
Monitoring Strategy

This makes the output suitable for business decision-makers.

18. Data Layer

The system stores data in structured databases.

Forecast Database

Stores:

product_id
forecast_demand
std_dev
lead_time
current_stock

Additional computed metrics:

volatility
reorder point
inventory gap
risk score

Vector Database (RAG)

Used for knowledge retrieval.

Pipeline:

PDF Upload
Text Extraction
Chunking
Embedding Generation
Vector Storage

Technologies used:

FAISS / Chroma

19. Reporting and Output

The system generates business-ready outputs.

PDF Reports

Generated using ReportLab.

Reports include:

- demand forecasts
- inventory alerts
- AI insights
- risk rankings

Email Notifications

Reports can be automatically emailed to stakeholders.

20. Deployment Architecture

NIYOJAN is deployed as a cloud-based system.

Containerization

Docker is used to package system components.

Docker containers include:

Frontend container
Backend container
Database volume

Cloud Deployment

The system is deployed on AWS EC2 using an Ubuntu-based instance.

Advantages:

- scalability
 - reproducibility
 - production-level deployment
-

21. Evaluation

The system is evaluated at two levels.

Forecast Accuracy

Metrics used:

MAE
RMSE

Results show that the LSTM forecasting model significantly outperforms baseline models.

Decision Quality

Decision accuracy evaluation shows approximately 90% correctness across risk categories.

System-Level Evaluation

The platform demonstrates:

- improved decision interpretability
 - strong usability for planners
 - reliable end-to-end automation
-

22. Limitations

Despite its strengths, the system has some limitations:

- static decision thresholds
 - limited dataset size
 - no automated model retraining pipeline
 - limited multi-product optimization
-

23. Future Work

Future improvements include:

- adaptive decision rules
- reinforcement learning planning agents
- real-time data ingestion pipelines
- multi-product optimization algorithms
- edge AI deployment

24. Conclusion

NIYOJAN demonstrates that effective demand planning requires more than accurate forecasting.

By combining machine learning, explainable decision logic, generative AI insights, and agentic AI planning, the system bridges the gap between prediction and actionable decision-making.

The platform evolves from a forecasting tool into a strategic AI planning system capable of simulation, reasoning, and executive decision support.

With its modular architecture, cloud deployment, and real-world usability, NIYOJAN represents a significant step toward intelligent, AI-driven supply chain planning systems.

If you want, I can also help you create a much stronger version of this report (the research-paper level version) that professors usually expect for AI/ML major projects and publications.